EPSR-test-ex57 fluctuations: three sphere sizes, three chained runs

Program(s): fluctuations | chained passes: 3 | verification: ALL IDENTICAL (all 3 passes)

What this case tests

Exercises `fluctuations` [1], which samples the number N of molecule centres in spheres of radius `rmax` placed at random positions in the box, the distribution of N - <N>, and the averages <N>, <N²> and S(0) = (<N²> - <N>²)/<N>, the long-wavelength limit of the structure factor, related to the isothermal compressibility [2]. Here three sphere sizes are accumulated over three chained runs, and the random-number state is carried from run to run.

Code exercised

Fortran (original, fortran_src/):

- `fluctuations.f`: `run_fluctuations` (averages, S(0), list-directed real*8 output), `find_fluctuations` (random spheres, counting)
- `auxiliary_routines.f`: `initran1`, `ran1`, `saveran1` (generator state in `randomno.dat`)
- `read_and_save_files.f`: `outgofz`, `save_files_others`; `setup_symbols_generic.f`: `setup_symbols_fluctuations`

Rust (port, rust/epsr-core/src/):

- `tools/fluctuations.rs`: `run`, `find_fluctuations`
- `rng.rs`: `Ran1::ran1`; `tools/anal_common.rs`: `initran1_file`, `saveran1_file`, `outgofz`
- `tools/anal_common.rs`: `set_sv_dble`; `symbols.rs`: `setup_input_file`, `read_input_file`, `save_inp_file`; `output.rs`: `r_int`

Physical system

Analysed configuration: `200DMSO400Water.ato`, an unchanged copy of the EPSR-test box of 200 dimethyl sulphoxide (DMSO; atom types S, O, C, H) and 400 water molecules (O1, H1): 3200 atoms, DMSO mole fraction 1/3, 300 K, cubic box of side 33.1213 Å (0.0881 atoms/Å³), the EPSR model of the deuterated DMSO-water neutron data (ISIS SANDALS runs SLS03863, SLS03871, SLS03850), cf. [3]. Spheres of radius 4, 6.5 and 10 Å, 3000 random centres each (Park-Miller generator with Bays-Durham shuffle [4,5]). After the three passes the reference gives <N> = 4.454, 18.974 and 69.158 molecules and S(0) = 0.634, 0.752 and 0.938.

Inputs

`numfluct.FLUCTUATIONS.dat`: `nsize 60`, `nsumt 0`, `nsamples 3000`, three sizes (`rmax` 4.0, 6.5, 10); no `randomno.dat` initially; `200DMSO400Water.ato` copied unchanged from EPSR-test.

How the accuracy was verified

`commands.txt` (1 program call, repeated 3 times, each pass reading the files written by the previous one) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; no masking except where noted below). The Fortran output is stored in `fortran_reference.tar.gz`. Result of the final regression run of the complete port: every file identical in all 3 passes.
`repeat 3`: pass 1 resets the generator (no `randomno.dat`) and saves its state; passes 2 and 3 continue from `randomno.dat` and average. Compared after each pass: `numfluct.FLUCTUATIONS.n01` (accumulated N - <N> distributions), the rewritten `.dat` file with `nsumt` and the running real*8 averages `ntotal`, `nsqtotal`, `szero` written list-directed (reference after pass 3: `nsumt 3`, `ntotal 4.4543333336909612`), and `randomno.dat`. The configuration is the same in every pass, so the chained passes test the read-back of the accumulated file and the running-average arithmetic.

References

[1] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).
[2] J.-P. Hansen and I. R. McDonald, Theory of Simple Liquids, 3rd ed., Academic Press (2006).
[3] A. K. Soper and A. Luzar, "A neutron diffraction study of dimethyl sulphoxide-water mixtures", J. Chem. Phys. 97, 1320-1331 (1992).
[4] S. K. Park and K. W. Miller, "Random number generators: good ones are hard to find", Commun. ACM 31, 1192-1201 (1988).
[5] W. H. Press, S. A. Teukolsky, W. T. Vetterling and B. P. Flannery, Numerical Recipes in Fortran 77, 2nd ed., Cambridge University Press (1992).
[6] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.